

## 20 Sortieren und Suchen



**Sortieren** und **Suchen** sind wohl die häufigsten Tätigkeiten in der Datenverarbeitung. Das kann man auch an der historischen Entwicklung sehen: die Vorläufer der heutigen Computer waren Sortiermaschinen für Lochkarten.

Die beiden Tätigkeiten Sortieren und Suchen hängen miteinander zusammen. In einem Sprichwort heißt es: „Wer Ordnung hält, ist nur zu faul zu suchen“. Frei übersetzt bedeutet es, dass die meisten Suchverfahren am besten dann funktionieren, wenn die Daten in sortierter Form vorliegen.

Unter Sortieren versteht man die Realisierung einer Ordnungsrelation innerhalb einer gegebenen Menge von Objekten.



Die Objekte können dabei in einer Liste, einer Tabelle oder in einem Array stehen. Im allgemeinen Fall können die Elemente auch unterschiedliche Längen haben.

Objekte werden sortiert, damit man später den Suchvorgang vereinfachen kann. Dabei kann das Sortierkriterium – die Ordnungsrelation – aus einem oder mehreren Schlüsseln bestehen.



So kann man beispielsweise eine Studentenliste zunächst nach Studiengängen sortieren und dann innerhalb der Studiengänge nach den Namen der Studenten suchen. Für das Sortieren existieren verschiedene Verfahren, die sich nach Aufwand, Speicherbedarf etc. unterscheiden.

Unter internen Sortiervorgängen versteht man Verfahren, bei denen der Sortiervorgang im Arbeitsspeicher durchgeführt wird. Externe Sortiervorgänge arbeiten auf Daten, die in Dateien auf einem externen Speicher gespeichert sind.



**Ergänzende Information** Die elektronische Version dieses Kapitels enthält Zusatzmaterial, auf das über folgenden Link zugegriffen werden kann [https://doi.org/10.1007/978-3-658-45209-4\\_20](https://doi.org/10.1007/978-3-658-45209-4_20).

Im Folgenden werden nur interne Sortier- und Suchverfahren betrachtet, also solche Verfahren, bei denen die Elemente in Arrays oder verketteten Listen (siehe Kapitel [→ 19](#)) im Arbeitsspeicher stehen. Wenn die Objekte sich nicht im Arbeitsspeicher, sondern in Dateien befinden, können im Prinzip die gleichen Algorithmen angewandt werden, sofern die technischen Voraussetzungen – beispielsweise wahlfreier Zugriff zu allen Objekten – für das jeweilige Verfahren gegeben sind.

In diesem Kapitel sind verschiedene Einsatzmöglichkeiten von Arrays (siehe Kapitel [→ 8.3](#)) sowie Anwendungen des **Rekursionsprinzips** (siehe Kapitel [→ 11.8](#)) zu finden.

Einige effiziente Such- und Sortierverfahren basieren auf Baumstrukturen und wurden bereits in Kapitel [→ 19.2.4](#) behandelt.

## 20.1 Interne Sortierverfahren

Im Folgenden sollen zwei grundsätzliche Algorithmen interner Sortierverfahren vorgestellt werden:

- Sortieren durch direktes Auswählen
- Sortieren mit dem Quicksort-Verfahren

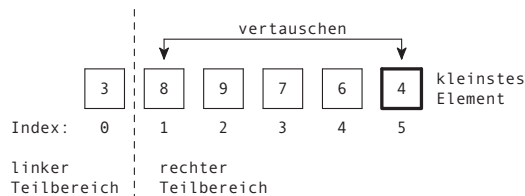
Um die Einführung in das Sortieren einfach zu gestalten, werden hier nur eindimensionale Arrays mit Integer-Werten nach aufsteigender Reihenfolge sortiert. Die Verschiebe- und Austauschoperationen werden direkt mit den Array-Elementen durchgeführt. Für den Vergleich der Werte genügen die Vergleichsoperatoren des C-Sprachumfanges. Die Verfahren können auf beliebige andere Schlüssel übertragen werden, wobei nur die entsprechenden Vergleichsrelationen vorhanden sein müssen.

### 20.1.1 Konzept für das iterative Sortieren durch direktes Auswählen

**Sortieren durch direktes Auswählen** wird auch als „**selection sort**“ oder – je nach Sortierrichtung – als „MinSort“ beziehungsweise „MaxSort“ bezeichnet und arbeitet nach folgendem Prinzip:

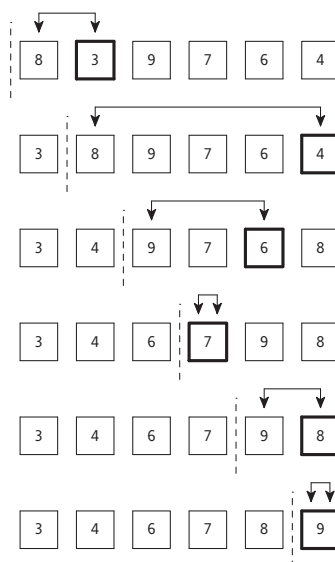
1. Teile das Array in zwei Bereiche: Ein linker, sortierter Teilbereich, der zu Beginn leer ist und ein rechter, unsortierter Teilbereich.
2. Suche im rechten unsortierten Teilbereich das Element mit dem kleinsten Wert.
3. Tausche das erste Element des rechten Teilbereichs mit dem gefundenen Element, das den kleinsten Wert hat.
4. Vergrößere den linken Bereich rechts um 1 Element und verkürze den rechten Bereich links um ein Element. Weiter bei Schritt 2.

Dieses Verfahren wird durch folgendes Bild veranschaulicht:



Das genannte Verfahren beginnt mit einem leeren linken Teilbereich, während der rechte Teilbereich zunächst das ganze Array enthält. Das Verfahren endet, wenn der rechte Teilbereich nur noch 1 Element enthält.

Das folgende Bild zeigt den gesamten Ablauf des Verfahrens:



### 20.1.2 Beispielprogramm für das Sortieren durch Auswählen

Im Folgenden wird als Beispiel ein C-Programm für das Sortieren durch Auswählen angegeben:

auswahl.c

```
#include <stdio.h>

void printArray(int* array, int anzahl, int schritt) {
    printf("Arbeitsschritt: %i  Array: ", schritt);
    for (int i = 0; i < anzahl; ++i)
        printf("%i ", array[i]);
    printf("\n");
}

int main(void) {

    int array[] = {8, 3, 9, 7, 6, 4};
    const int anzahl = sizeof(array) / sizeof(array[0]);

    printArray(array, anzahl, 0);

    // Schrittweise durch das gesamte Array
    for (int schritt = 0; schritt < anzahl - 1; ++schritt) {
        int minPos = schritt;

        // Suche nach kleinstem Element
        for (int pos = schritt + 1; pos < anzahl; ++pos) {
            if (array[pos] < array[minPos]) {
                minPos = pos;
            }
        }

        int swap = array[minPos];
        array[minPos] = array[schritt];
        array[schritt] = swap;

        printArray(array, anzahl, schritt + 1);
    }
    return 0;
}
```

Beispiel für den Programmablauf:

```
Arbeitsschritt: 0  Array: 8 3 9 7 6 4  
Arbeitsschritt: 1  Array: 3 8 9 7 6 4  
Arbeitsschritt: 2  Array: 3 4 9 7 6 8  
Arbeitsschritt: 3  Array: 3 4 6 7 9 8  
Arbeitsschritt: 4  Array: 3 4 6 7 9 8  
Arbeitsschritt: 5  Array: 3 4 6 7 8 9
```

### 20.1.3 Konzept für das rekursive Sortieren mit dem Quicksort-Verfahren

Das **Quicksort**-Verfahren wurde von Hoare [Hoa62] entwickelt. Es ist ein effizientes Sortierverfahren und ein gutes Beispiel für die Anwendung eines rekursiven Algorithmus. Der Algorithmus wird hier nur in der einfachsten Form dargestellt.

Das Verfahren arbeitet nach dem Prinzip „**teile und herrsche**“ (auf Englisch „**divide and conquer**“): Man teilt das Array in zwei Teile – oder genauer gesagt – in zwei Teilarrays auf.

Für das rekursive Sortieren mit dem Quicksort-Verfahren wird ein beliebiges Vergleichselement aus dem Array benötigt. Es bietet sich an, das mittlere Element des Arrays als Vergleichselement zu wählen. Die Bedingung an die Teilarrays ist dann, dass das linke Teilarray nur solche Elemente hat, die kleiner oder gleich wie das Vergleichselement sind und das rechte Teilarray hat nur solche Elemente, die größer oder gleich sind wie das Vergleichselement. Diese Teilarrays sind relativ einfach durch Vergleich und Austausch herzustellen. Danach wird das gleiche Teilungsverfahren auf das jeweilige linke und rechte Teilarray rekursiv angewandt, bis jedes Teilarray weniger als 2 Elemente hat.



Das Verfahren hat also folgende Schritte:

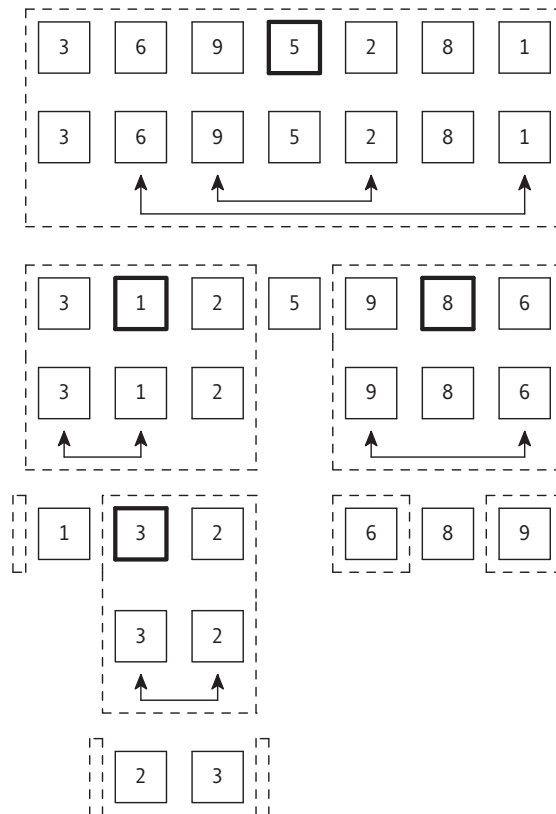
1. Auswahl eines beliebigen Vergleichswertes, beispielsweise derjenige des mittleren Elements des Arrays:

```
vergleichsWert = array[(startLinks + startRechts) / 2];
```

Dabei stehen `startLinks` und `startRechts` für den äußerst linken und äußerst rechten Index des Arrays, am Anfang also für 0 und  $n - 1$ .

2. Linkes Teilarray mit kleineren (oder gleichen) und rechtes Teilarray mit größeren (oder gleichen) Elementen erzeugen.
  - a) Absuchen des linken Arrays von links her mittels einer Laufvariablen `laufLinks`, bis ein Element mit größerem Wert als der Vergleichswert gefunden wird. `laufLinks` ist zu Beginn gleich `startLinks` und wird in jedem Schritt inkrementiert.
  - b) Absuchen des rechten Arrays von rechts her mittels einer Laufvariable `laufRechts`, bis ein Element mit kleinerem Wert als der Vergleichswert gefunden wird. `laufRechts` ist zu Beginn gleich `startRechts` und wird in jedem Schritt dekrementiert.
  - c) Vertauschen des gefundenen größeren Elements an `laufLinks` mit dem gefundenen kleineren Element an `laufRechts`.
  - d) Inkrementieren von `laufLinks` und dekrementieren von `laufRechts` um jeweils 1.
  - e) Schritte 2.a bis 2.d wiederholen, bis sich die beiden Suchindizes überkreuzen. Die Kreuzung definiert die Stelle, an der das Array in Schritt 3 in Teilarrays zerteilt wird.
3. Rekursive Zerlegung des linken und rechten Teilarrays. Das neue linke Teilarray besteht aus den Elementen `[startLinks, laufRechts]` und das neue rechte Teilarray aus den Elementen `[laufLinks, startRechts]`. Die Schritte 1 und 2 werden nun mit diesen beiden Teilarrays erneut durchgeführt.
4. Die Rekursion erfolgt solange, bis die Teilarrays weniger als 2 Elemente haben. Teilarrays mit einem einzigen Element sind trivialerweise sortiert.

Nach Ende von Schritt 2 stehen im linken Teilarray nur Elemente mit Werten, die kleiner als der Wert des Vergleichselements sind, und im rechten Teilarray nur Elemente, deren Werte größer als der Wert des Vergleichselements sind. Das folgende Bild symbolisiert den Ablauf des Verfahrens anhand eines Beispiels:



Die dick umrandeten Felder im obigen Bild stellen die ausgewählten Vergleichselemente dar. Die gestrichelten Felder zeigen die Teilarrays, welche noch sortiert werden müssen.

Das Vergleichselement bei den gegebenen Beispielzahlen {3, 6, 9, 5, 2, 8, 1} ist wegen der in Verfahrensschritt (1) beschriebenen Vorschrift die Zahl 5. Das Array wird nun in zwei Teilarrays {3, 1, 2} und {9, 8, 6} geteilt, wobei im linken Teilarray alle Zahlen kleiner als 5, im rechten alle größer als 5 sind. Das Vergleichselement mit dem Wert 5 ist tatsächlich nicht in den Teilarrays enthalten, da durch die Schritt-Variablen über Elemente hinwegwandern, welche gleich sind wie das Vergleichselement. Das Vergleichselement befindet sich nach diesem Schritt somit bereits an seiner endgültigen Position.

Als nächstes wird dieser Zerlegungsvorgang rekursiv auf das linke Teilarray {3, 1, 2} angewandt. Dieses wird mit der Vergleichszahl 1 durch Vertauschen in ein leeres Teilarray {} und das Teilarray {3, 2} geteilt. Das leere Teilarray entsteht, da die Laufvariable beim Vergleich über das Vergleichselement hinwegwandert. Dieser besondere Umstand muss unbedingt in der Implementation abgefangen werden. Das Teilarray {3, 2} schlussendlich wird nochmals rekursiv weiterbehandelt. Nach nur einem weiteren Schritt bleiben nur leere Teilarrays übrig. Somit ist dieses Teilarray vollständig sortiert.

Das rechte Teilarray {9, 8, 6} wird mit dem Vergleichselement 8 entsprechend so sortiert, dass links das Teilarray {6} und rechts das Teilarray {9} übrigbleibt. Da Arrays mit nur einem Element automatisch sortiert sind, ist damit der Sortiervorgang abgeschlossen.

#### 20.1.4 Beispielprogramm für das Sortieren mit dem Quicksort-Algorithmus

Im Folgenden wird als Beispiel eine C-Funktion für das Sortieren mit dem Quicksort-Algorithmus angegeben:

quicksort.c

```
#include <stdio.h>

void printArray(int* array, int start, int ende) {
    for (int i = 0; i < start; ++i)
        printf(" ");

    for (int i = start; i <= ende; ++i)
        printf("%i ", array[i]);

    printf("\n");
}

void zerlege(int* array, int startLinks, int startRechts) {
    int laufLinks = startLinks;
    int laufRechts = startRechts;
    int vergleichsWert = array[(startLinks + startRechts) / 2];

    do {
        while (array[laufLinks] < vergleichsWert)
            ++laufLinks;

        while (array[laufRechts] > vergleichsWert)
            --laufRechts;
```



```

        if (laufLinks <= laufRechts) {
            int swap = array[laufLinks];
            array[laufLinks] = array[laufRechts];
            array[laufRechts] = swap;
            ++laufLinks;
            --laufRechts;
        }
    } while (laufLinks <= laufRechts);

    printArray(array, startLinks, startRechts);

    // Teilarrays rekursiv sortieren
    if (startLinks < laufRechts)
        zerlege(array, startLinks, laufRechts);

    if (laufLinks < startRechts)
        zerlege(array, laufLinks, startRechts);
}

void quickSort(int* array, int n) {
    zerlege(array, 0, n - 1);
}

int main(void) {
    int array[] = {7, 3, 8, 6, 9, 2, 1, 4};
    int numInArray = sizeof(array) / sizeof(int);

    printArray(array, 0, numInArray - 1);
    quickSort (array, numInArray);
    printArray(array, 0, numInArray - 1);

    return 0;
}

```

Beispiel für den Programmablauf:

```

7 3 8 6 9 2 1 4
4 3 1 2 9 6 8 7
2 1 3 4
1 2
    3 4
        6 9 8 7
            7 8 9
1 2 3 4 6 7 8 9

```

### 20.1.5 Sortiervverfahren im Vergleich



Wenn es mehrere Verfahren zur Lösung des gleichen Problems gibt, stellt sich automatisch die Frage, welches das „bessere“ Verfahren ist. Genauso verhält es sich mit den Sortiervverfahren. In der Informatik werden diese Fragestellungen unter dem Stichwort „**Komplexitätsbetrachtungen**“ untersucht. Dabei wird ein Verfahren beziehungsweise ein Algorithmus im Hinblick auf seinen Speicherbedarf oder die benötigte Rechenzeit analysiert. Die Ergebnisse erlauben dann einen Vergleich der Verfahren.

Die hier vorgestellten Sortiervverfahren sollen bezüglich der benötigten Rechenzeit miteinander verglichen werden. Dazu dienen die folgenden Vorüberlegungen, die Einfluss auf die Rechenzeit haben:

Grundsätzlich werden für jedes Sortiervverfahren Vergleichs- und Austauschoperationen benötigt. Der Aufwand wird zunächst immer für die Anzahl der Vergleiche ( $V$ ) sowie die Anzahl der Bewegungen ( $B$ ) von Elementen berechnet, wobei von einer Anzahl von  $n$  unsortierten Elementen ausgegangen wird.



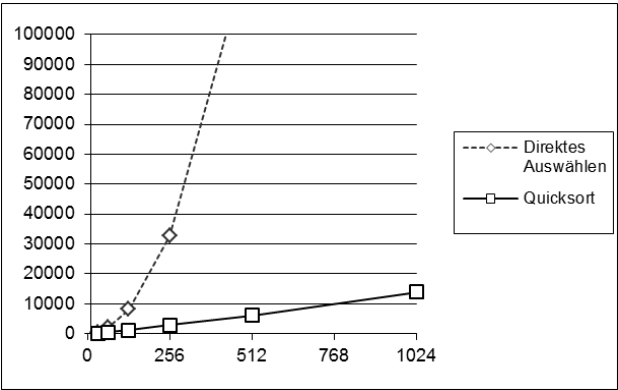
Komplexitätsbetrachtungen sind vor allem bei einer größeren Anzahl von Elementen wichtig. Dabei geht es insbesondere darum, wie sich der Aufwand des Verfahrens in Abhängigkeit zu der Anzahl der Elemente  $n$  verhält. In der Informatik wird hierfür die sogenannte **O-Notation** verwendet. Man sagt beispielsweise: Ein Suchverfahren hat eine Laufzeit von  $O(n^2)$ . Das bedeutet, dass die Laufzeit mit wachsendem  $n$  quadratisch ansteigt. Für 1000 Elemente benötigt ein solches Verfahren somit 1 Million Schritte. Ein Verfahren mit  $O(n)$  hingegen benötigt für 1000 Elemente nur 1000 Schritte. Somit ergibt sich eine klare Aussage:  $O(n^2)$  ist langsamer als  $O(n)$ .

Die beiden dargestellten Sortiervverfahren „Sortieren durch direktes Auswählen“ und „Quicksort“ werden hier bezüglich des Aufwandes für Vergleiche und für Bewegungen einander gegenübergestellt. Anzumerken ist, dass es weitere Sortiervverfahren wie beispielsweise „Bubblesort“, „Sortieren durch direktes Einfügen“ und „Shellsort“ gibt, die bezüglich des Aufwandes schlechter sind als „Quicksort“. Dabei wird als Ausgangssituation für die Gegenüberstellung ein unsortiertes, mit zufällig erzeugten Zahlen belegtes Array gewählt. Als Ergebnis wird die Anzahl der Vergleiche und die Anzahl der Bewegungen für verschieden große Arrays dargestellt.

Die folgende Tabelle vergleicht den Aufwand für das Sortieren für verschieden große Arrays. Es werden dabei einfach alle Operationen gezählt, welche entweder einen Vergleich ausführen, oder aber eine Bewegung.

| Anzahl der<br>Arrayelemente |            | 32  | 64   | 128  | 256   | 512    | 1024   |
|-----------------------------|------------|-----|------|------|-------|--------|--------|
| Direktes Auswählen          | Vergleiche | 496 | 2016 | 8128 | 32640 | 130816 | 523776 |
|                             | Bewegungen | 163 | 366  | 792  | 1781  | 4077   | 8700   |
| Quicksort                   | Vergleiche | 195 | 499  | 1198 | 2882  | 6120   | 13885  |
|                             | Bewegungen | 163 | 342  | 824  | 1910  | 4140   | 8809   |

Das folgende Bild veranschaulicht diese Resultate:



Es ist deutlich zu sehen, dass das Quicksort-Verfahren dem anderen weit überlegen ist. Selbst für große Arrays steigt die Laufzeit nur langsam an.

Tatsächlich hat das gesamte Sortierverfahrenverfahren eine gemittelte Laufzeit von  $O(n \log_2(n))$ .

Im Gegensatz dazu steigt der mittlere Aufwand des einfachen Sortierverfahrens (direktes Auswählen) quadratisch mit der Anzahl der Elemente an:  $O(n^2)$

Für eine große Anzahl von Elementen sollte daher immer eines der besseren Sortierverfahren wie Quicksort verwendet werden.

Es ist zu beachten, dass Quicksort unter bestimmten Bedingungen ebenfalls eine schlechte Laufzeit von  $O(n^2)$  aufweisen kann. Man geht jedoch von der optimistischen Annahme aus, dass das gewählte Element das Array in zwei möglichst gleich große Hälften teilt. Geht man von einer zufälligen Anordnung der Elemente aus, so steigt bei zufälliger Wahl des Vergleichselements der durchschnittliche Aufwand an Tauschoperationen gegenüber dem günstigsten Fall tatsächlich nur um den Faktor  $2 \ln 2 = 1.39$  an (siehe [Hoa62]). Dies ist keine wesentliche Verschlechterung des optimistischen Erwartungswertes. Deswegen wird Quicksort im Allgemeinen stets mit der Laufzeit  $O(n \log_2(n))$  angegeben.

Wenn allerdings zufälligerweise immer das größte oder das kleinste Element als Teilungselement gewählt wird, so wird Quicksort zu einem extrem langsamen Verfahren.



Bei der Zerlegung eines Arrays mit  $n$  Elementen würden in diesem Fall jeweils ein Teilarray mit  $n - 1$  Elementen und ein Teilarray mit nur einem einzigen Element entstehen. Dadurch wird der Aufwand an Zerlegungen und die Rekursionstiefe extrem groß, da jeweils Teilarrays der Länge 1 entstehen.

Um das schlechte Verhalten im Extremfall zu vermeiden, sollte das Quicksort-Verfahren bezüglich der Wahl des Vergleichselements optimiert werden. Hierzu gibt es Vorschläge für eine relativ gute Wahl des Vergleichselements.



Ein Vorschlag ist beispielsweise, dass man 3 Elemente zufällig aussucht und das mittlere der 3 Elemente als Vergleichselement nimmt.

### 20.1.6 Die Quicksort-Funktion der Standardbibliothek

Die Funktion `qsort()` aus der Standard-Bibliothek von C (Prototyp in `<stdlib.h>`) enthält eine optimierte Form des Quicksort-Algorithmus mit einer beliebigen Vergleichsfunktion, die mit Hilfe eines Funktionszeigers übergeben wird.



Der Prototyp ist wie folgt aufgebaut:

```
void qsort(  
    void* base,  
    size_t nmemb,  
    size_t size,  
    int (*compar)(const void*, const void*));
```

Die 4 Parameter der Funktion `qsort()` haben die folgende Bedeutung:

- `base`: Pointer auf das erste Element des Arrays, also in der Regel der Arrayname.
- `nmemb`: Anzahl der Elemente im Array.
- `size`: Größe eines Array-Elements in Bytes (beispielsweise mittels `sizeof`).
- `compar`: Funktionsname einer existierenden, selbst geschriebenen Vergleichsfunktion, die als Parameter zwei Pointer auf `void` übergeben bekommt, die auf zwei zu vergleichende Elemente aus dem Array zeigen. Die Funktion muss diese beiden Elemente vergleichen und das Ergebnis des Vergleichs als `int`-Zahl wie folgt zurückgeben: Falls beide Elemente gleich sind, so wird eine 0 zurückgegeben. Ist das erste Element größer als das zweite Element, wird eine Zahl größer 0 zurückgegeben und im umgekehrten Fall eine Zahl kleiner 0.

Die `qsort()`-Funktion ruft im Zuge des intern ablaufenden Sortiervorganges immer wieder die Vergleichsfunktion auf und übergibt ihr als Parameter zwei Pointer auf zwei zu vergleichende Elemente.



Dadurch ist es möglich, die `qsort()`-Funktion sehr flexibel einzusetzen und beliebige Daten zu sortieren, da die Funktion sich nicht um den Vergleich an sich kümmern muss. Beim Programmieren der Vergleichsfunktion wird ja vorgegeben, was größer, kleiner oder gleich ist. Dies muss dann für unterschiedliche Datentypen eben mit verschiedenen Vergleichsfunktionen erfolgen.

Die Pointer auf `void` können in der Vergleichsfunktion in einen entsprechenden Pointer-Typ gewandelt werden (siehe Kapitel [8.2](#)), damit der Vergleich durchgeführt werden kann. Dieser Pointer-Typ muss so gewählt werden, dass mittels einer Dereferenzierung der Zugriff auf die zu vergleichenden Objekte möglich wird.

Folgendes ist ein Beispiel:

`qsort.c`

```
#include <stdlib.h>
#include <stdio.h>

int aufsteigend(const void* a, const void* b) {
    return (*(int*)a - *(int*)b);
}

int absteigend(const void* a, const void* b) {
    return (*(int*)b - *(int*)a);
}

void printArray(int* array, int anzahl) {
    for (int i = 0; i < anzahl; ++i)
        printf("%i ", array[i]);
    printf("\n");
}

int main(void) {
    int array[] = {7, 3, 8, 6, 9, 2, 1, 4};
    int anzahl = sizeof(array) / sizeof(int);

    printArray(array, anzahl);
    qsort(array, anzahl, sizeof(int), aufsteigend);
    printArray(array, anzahl);
    qsort(array, anzahl, sizeof(int), absteigend);
    printArray(array, anzahl);
}
```

Ausgabe:

```
7 3 8 6 9 2 1 4
1 2 3 4 6 7 8 9
9 8 7 6 4 3 2 1
```

Um die `qsort()`-Funktion aus der Standardbibliothek zu nutzen, muss nur eine solche Vergleichsfunktion für die Elemente des Arrays implementiert werden. Die Funktionalität des Quicksort-Verfahrens steht also über die Standardbibliothek ohne großen Entwicklungsaufwand zur Verfügung.

20.2 Einfache Suchverfahren

Eine häufige Anwendung in der Informationsverarbeitung ist das **Suchen** eines Objektes in einer Menge von Objekten gleichen Typs, wie zum Beispiel das Suchen der Daten eines bestimmten Mitarbeiters in den Personaldaten. Die folgende Tabelle symbolisiert irgendwelche Personaldaten:

| Personalnummer | Name   | Vorname  | Abteilung |
|----------------|--------|----------|-----------|
| 125            | Müller | Hans     | EK1       |
| 015            | Sommer | Elke     | FE        |
| 093            | Braun  | Charly   | FE        |
| 007            | Bond   | Johannes | SEC       |
| ...            |        |          |           |

Ein Element umfasst die Datenfelder Personalnummer, Name, Vorname und Abteilung. Eine derartige Tabelle kann man im Arbeitsspeicher in Form eines Arrays aus Strukturen implementieren.

Die Tabelle soll dabei beliebig durchsucht werden können, beispielsweise nach dem Vornamen „Charly“ oder der Personalnummer 007. Das Suchen wird beendet, wenn das gesuchte Element gefunden oder das Ende der Tabelle erreicht wurde.

Der Suchbegriff beziehungsweise **Schlüssel** ist Teil eines Objektes.



Die Objekte (Elemente) können dabei in Listen, Arrays oder Dateien stehen, wobei im allgemeinen Fall die Objekte unterschiedliche Größen haben dürfen.

Beim Suchen kommt es dem Anwender darauf an, dass die gesuchten Elemente schnell gefunden werden.



Hierbei unterscheidet man grundsätzlich zwischen dem Suchen in unsortierten Mengen und dem Suchen in sortierten Mengen.

### 20.2.1 Unsortierte Menge: Sequenzielles Suchen

Falls die Elemente einer Menge nicht sortiert sind, kann nur sequenziell gesucht werden, sprich jedes Element muss einzeln geprüft werden.

Der Algorithmus „**Sequenzielles Suchen**“ besteht aus dem Vergleich des Suchbegriffes mit den Schlüsselwerten der Elemente einer ungeordneten Menge, die beispielsweise in Form eines Arrays oder einer verketteten Liste vorliegen kann.



Sequenzielles Suchen ist eine ineffiziente Suchmethode, da im ungünstigsten Falle alle  $n$  Elemente der Menge mit dem Suchbegriff verglichen werden müssen.



Im Mittel sind bei diesem Algorithmus  $n/2$  Suchschritte erforderlich.



Die Effizienz eines solchen Verfahrens kann gesteigert werden, wenn die am häufigsten gesuchten Elemente an den Anfang des Arrays geschrieben werden. In diesem Fall muss allerdings die Suchwahrscheinlichkeit bekannt sein und die Menge vorher entsprechend aufbereitet werden.

Bei einer relativ kleinen Menge kann je nach Situation der Aufwand von sequenziellem Suchen noch akzeptabel sein. Ab einer gewissen Größe jedoch wird man auf effizientere Algorithmen zählen müssen.



### 20.2.2 Sortierte Menge: Halbierungssuchen

Voraussetzung für das Halbierungssuchen, welches auch unter dem Begriff „**Binäres Suchen**“, oder auf Englisch „**binary search**“ bekannt ist, ist eine sortierte Menge von Elementen.



Um eine Menge zu sortieren, können verschiedene Sortierverfahren angewendet werden. Zwei Beispiele wurden in Kapitel [→ 20.1](#) gegeben.

Im Folgenden wird zur Vereinfachung ein Array bestehend aus `int`-Werten betrachtet.

Beim Halbierungssuchen wird wie folgt vorgegangen:

1. Zu Beginn ist das gesamte Array das Suchintervall.
2. Das Element in der Mitte des Intervalls wird geprüft.
3. Entspricht das Element in der Mitte dem Suchkriterium, so ist das Element gefunden und der Algorithmus ist beendet.
4. Ist der Schlüssel des mittleren Elements größer als der gesuchte Schlüssel, so muss sich das Element im unteren Teilintervall befinden. Ist er kleiner, so muss es sich im oberen Teilintervall befinden. Die Intervallgrenzen werden entsprechend angepasst und bei Schritt 2 fortgefahren.
5. Die Halbierungen der Teilintervalle werden solange fortgesetzt, bis das Element gefunden wurde oder das Teilintervall leer ist. Im letzteren Falle ist das gesuchte Element nicht in der Liste.

Das Verfahren des Halbierungssuchens arbeitet korrekt für beliebige  $n$ , der Algorithmus hat die Komplexität  $O(\log_2(n))$ .



Bei 1000 Elementen wird somit mit diesem Verfahren das gesuchte Element nach höchstens rund 10 Suchschritten ( $\log_2(1000) \approx 10$ ) gefunden, während bei sequenziellem Suchen im Mittel 500 Suchschritte notwendig gewesen wären.

### 20.2.3 Beispiel für das Halbierungssuchen

Im folgenden Beispiel für das Halbierungssuchen (binäres Suchen) liegt ein geordnetes Array mit int-Elementen vor.

binaerSuchen.c

```
#include <stdio.h>

int binaer_suchen(int suchbegriff, const int* array, int anzahl) {
    int low = 0;
    int high = anzahl - 1;
    int mid;

    while (low <= high) {
        mid = (low + high) / 2;
        printf("Suche in [%d, %d] an Index %d: %d\n",
            low, high, mid, array[mid]);

        if (suchbegriff < array[mid])
            high = mid - 1;
        else if (suchbegriff > array[mid])
            low = mid + 1;
        else
            return mid; // gefunden
    }
    return -1; // nicht gefunden
}

int main(void) {
    int array[] = {2, 5, 11, 17, 23, 31, 33, 37, 44, 50, 59, 62};
    int anzahl = sizeof(array) / sizeof(int);

    int suchbegriff = 23;
    int index = binaer_suchen(suchbegriff, array, anzahl);
    if (index == -1)
        printf("Nicht gefunden \n");
    else
        printf("Element mit Index %d = %d\n", index, suchbegriff);

    return 0;
}
```

Hier die Ausgabe des Programms:

```
Suche in [0, 11] an Index 5: 31
Suche in [0, 4] an Index 2: 11
Suche in [3, 4] an Index 3: 17
Suche in [4, 4] an Index 4: 23
Element mit Index 4 = 23
```

Im ersten Schritt wird der Index 5 als Mitte bestimmt. Das gefundene Element 31 ist größer als 23, deswegen wird das tiefere Intervall  $[0, 4]$  durchsucht. Im nächsten Schritt befindet sich die Mitte an Index 2, das Element ist jedoch kleiner als 23, also wird das obere Teilintervall  $[3, 4]$  genommen. Im dritten Schritt ist das mittlere Element ebenfalls kleiner als 23, weswegen schlussendlich das Intervall  $[4, 4]$  durchsucht wird. Wenn das zu durchsuchende Intervall nur aus einem Element besteht, dann muss dieses entweder das gesuchte Element sein oder aber das Element ist nicht in der Menge. Hier entspricht das Element tatsächlich der Zahl 23 und der Algorithmus gibt den gefundenen Index 4 zurück.

#### 20.2.4 Beispielprogramm mit `bsearch()`

Genauso wie es für das Quicksort-Verfahren eine Funktion in der Standardbibliothek `stdlib.h` gibt (vergleiche `qsort()` in Kapitel [→ 20.1.6](#)), bietet diese Bibliothek auch eine Implementation der binären Suche an unter dem Namen `bsearch()`:

```
void* bsearch(
    const void* key,
    const void* base,
    size_t nmemb,
    size_t size,
    int (*compar)(const void*, const void*));
```

Ähnlich wie bei der Funktion `qsort()` wird beim Parameter `base` der Pointer auf das zu durchsuchende Array angegeben, der Parameter `nmemb` definiert, wie viele Elemente sich in dem Array befinden und mit `size` wird die Größe eines einzelnen Elements angegeben. Die Vergleichsfunktion `compar()` funktioniert genau gleich wie bei `qsort()`. Als zusätzlichen ersten Parameter erwartet die Funktion `bsearch()` einen Pointer `key` auf den Schlüssel, nach dem gesucht werden soll.

Wenn der Schlüssel in dem Array nicht gefunden wird, so wird ein NULL-Pointer zurückgegeben. Wenn er jedoch gefunden wird, so wird ein Pointer auf den Eintrag im Array zurückgegeben, innerhalb welchem der Schlüssel zu finden ist. Wenn derselbe Schlüssel an mehreren Stellen im Array zu finden ist, ist undefiniert, welche der möglichen Stellen zurückgegeben wird.

Folgendes Beispiel verdeutlicht die Verwendung der Funktion `bsearch()`:

`bsearch.c`

```
#include <stdlib.h>
#include <stdio.h>

int ordnung(const void* a, const void* b) {
    return (*(int*)a - *(int*)b);
}

int main(void) {
    int array[] = {2, 5, 11, 17, 23, 31, 33, 37, 44, 50, 59, 62};
    int anzahl = sizeof(array) / sizeof(int);

    int key = 23;
    int* keyptr = bsearch(&key, array, anzahl, sizeof(int), ordnung);

    if (keyptr)
        printf("Gefunden an Index %d\n", (int)((int*)keyptr - array));
    else
        printf("Nicht gefunden\n");
}
```

Die Ausgabe lautet:

```
Gefunden an Index 4
```

## 20.3 Suchen nach dem Hashverfahren

Beim binären Suchen wurde der Aufwand gegenüber dem sequenziellen Suchen von  $O(n)$  auf  $O(\log_2(n))$  reduziert. Bei 1000 Elementen werden somit bei binärem Suchen immer noch 10 Schritte benötigt. Wünschenswert wäre es, mit ein bis zwei Schritten direkt ein gesuchtes Element zu finden. Man spricht dann von einem  $O(1)$ -Algorithmus.

Mit einem bis zwei Schritten direkt ein gesuchtes Element zu finden, ist näherungsweise erreichbar, wenn aus dem Suchbegriff direkt durch Berechnung auf die Position im Array geschlossen werden kann.



Solche Verfahren werden „Hashverfahren“ genannt, aus Gründen, die noch erklärt werden.

### 20.3.1 Einführendes konzeptionelles Beispiel

Es seien die vier Zahlen  $\{2, 4, 8, 9\}$  gegeben. Diese Zahlen werden in einem Array mit 10 Elementen so gespeichert, dass die jeweilige Zahl  $i$  an der Position  $i$  in dem Array gespeichert wird. Die anderen, unbenutzten Positionen des Arrays werden durch eine „Frei“-Markierung  $-1$  als unbesetzt gekennzeichnet.

|        |    |    |   |    |   |    |    |    |   |   |
|--------|----|----|---|----|---|----|----|----|---|---|
|        | -1 | -1 | 2 | -1 | 4 | -1 | -1 | -1 | 8 | 9 |
| Index: | 0  | 1  | 2 | 3  | 4 | 5  | 6  | 7  | 8 | 9 |

Wenn nun ein Suchbegriff  $x$  vorgelegt wird (Zahl zwischen 0 und 9), so kann man durch Vergleich der Zahl  $x$  mit dem Element an Position  $x$  des Arrays feststellen, ob die Zahl sich im Array befindet oder dort nicht enthalten ist.

### 20.3.2 Einfache Schlüsseltransformation und Konflikte

Die in dem einführenden Beispiel dargestellte Methode lässt sich für beliebige Schlüssel in der im Folgenden beschriebenen Form sinnvoll erweitern.

Es sollen  $n$  Elemente mit beliebigen **Schlüsseln** (hier der Einfachheit halber vom Typ `int`) in einer Tabelle gespeichert werden. Die Schlüssel seien beliebig verteilt über  $[0 \dots K]$ . Zur Speicherung der Elemente soll eine Tabelle mit  $M$  Elementen verwendet werden, wobei  $M$  größer ist als  $n$ . Aus dem Schlüssel wird nun die Position in der Tabelle berechnet. An einer Position der Tabelle kann in dem betrachteten Fall jeweils ein einziger Schlüsselwert gespeichert werden.

Zunächst sei angenommen, die Tabellenlänge  $M$  sei eine Primzahl. Die Position eines Elementes mit dem Schlüssel  $x$  in der Tabelle kann dann durch folgende Transformation ermittelt werden:

$$\text{Position} = x \bmod M$$

Damit können auch Schlüssel, die größer als  $M$  sind, in der Tabelle untergebracht werden.

Durch die Modulo-Funktion wird – bildlich gesprochen – ein Teil des Schlüssels abgehackt (auf Englisch „hashed“). Daher wird dieses Verfahren im Englischen als „**hash**“-Verfahren bezeichnet.



Angenommen,  $M$  sei deutlich größer als  $n$ . Wenn nun  $n$  Elemente in eine Tabelle mit  $M$  Plätzen an die nach obigem Algorithmus berechneten Positionen eingetragen werden, so werden die Elemente in der Tabelle mit Lücken gespeichert sein. Daher stammt auch die deutsche Bezeichnung „Streuspeicherung“ für das Verfahren.

Im folgenden Beispiel wird eine Tabelle mit  $M = 7$  Plätzen und  $n = 3$  Elementen gezeigt, deren Positionen über  $x \bmod M$  berechnet wurden:

| Position | Inhalt | Berechnung       |
|----------|--------|------------------|
| 0        | 77     | $77 \bmod 7 = 0$ |
| 1        |        |                  |
| 2        | 16     | $16 \bmod 7 = 2$ |
| 3        |        |                  |
| 4        |        |                  |
| 5        | 40     | $40 \bmod 7 = 5$ |
| 6        |        |                  |

Wie man sieht, sind die 3 Werte über die Tabelle gestreut, 4 Plätze der Tabelle sind noch frei. Zunächst wird angenommen, dass alle Schlüssel zu verschiedenen Positionen in der Tabelle führen. In einer derart organisierten Tabelle lässt sich nun ein Element sehr leicht finden, indem man mit dem Schlüssel des gesuchten Elementes die gleiche Transformation durchführt wie beim Eintragen eines Elementes in die Tabelle. Ein Vergleich mit dem Inhalt an der berechneten Tabellenposition zeigt nun, ob das gesuchte Element in der Tabelle enthalten ist oder nicht.

Leider stimmt die Annahme, dass alle Schlüssel zu verschiedenen Positionen führen, häufig nicht. Es tritt ein sogenannter Konflikt auf.



Fügt man obigem Beispiel noch die Zahl 23 hinzu, so ergibt  $23 \bmod 7$  den Index 2.

Somit gibt es zwei Zahlen (16 und 23), welche beide am Index 2 gespeichert werden sollten. Diese Situation wird als „**Konflikt**“ oder „**Kollision**“ bezeichnet. Dieser Konflikt muss beseitigt werden. Dazu existieren verschiedene Strategien, die im Weiteren noch erläutert werden.

### 20.3.3 Konfliktlösung mit linearer Sondierung

Zunächst wird eine einfache Konfliktstrategie erläutert, die auf einer linearen Sondierung besteht. Es wird dabei davon ausgegangen, dass das Array im Verhältnis zu der Anzahl an zu speichernden Werte sehr groß ist.

Bei der Konfliktlösung mit linearer Sondierung wird, wenn eine Position in der Tabelle schon besetzt ist und ein neuer Schlüssel zur gleichen Position in der Tabelle führt, ab der berechneten Position linear aufwärts nach der nächsten freien Position gesucht und das Element dort gespeichert. Bei dieser neuen Position muss natürlich ebenfalls mit modulo  $M$  sichergestellt werden, dass sie nicht die Größe der Tabelle sprengt.



Im obigen Beispiel werden die Konflikte dann in folgender Weise gelöst:

| Position | Inhalt | Berechnung                |
|----------|--------|---------------------------|
| 0        | 77     | $77 \bmod 7 = 0$          |
| 1        |        |                           |
| 2        | 16     | $16 \bmod 7 = 2$          |
| 3        | 23     | $23 \bmod 7 = 2 \dots +1$ |
| 4        |        |                           |
| 5        | 40     | $40 \bmod 7 = 5$          |
| 6        |        |                           |

Beim Suchen wird dann das gleiche Verfahren angewandt. Zunächst wird aus dem Suchschlüssel die Position berechnet und das gesuchte Element mit dem dort gespeicherten Element verglichen.

Beim Suchen ist bei Übereinstimmung der Position das Element gefunden, bei Nichtübereinstimmung wird ab der berechneten Position linear aufwärts das Element in der Liste gesucht (mit modulo  $M$ ), bis es gefunden wurde oder ein Platz frei ist. Wenn ein freier Platz gefunden wurde, ist das Element nicht in der Liste und die Suche ist beendet.





Es ist offensichtlich, dass dieses Verfahren bei häufigen Kollisionen sehr ineffizient wird. Die Kollisionswahrscheinlichkeit steigt mit dem Füllgrad der Liste  $n/M$  deutlich an.

Die Tabellenlänge muss vorher bekannt sein. Die Anzahl  $n$  darf  $M$  nie übertreffen. Generell sollte die Tabelle einen Füllgrad  $n/M$  von 80% bis 90% nicht übersteigen.



In diesem Verfahren wurden bei Kollisionen die neuen Positionen jeweils mit einer Erhöhung um die Schrittweite 1 berechnet. Man kann auch andere Schrittweiten, die größer als 1 und kleiner als die Tabellenlänge  $M$  sind, wählen, erhält aber dadurch keine Verbesserung des Verfahrens.

Das Löschen von Elementen ist ebenfalls möglich. Ein Element wird gelöscht, indem eine Löschmarkierung als Schlüssel in die Tabelle eingetragen wird. Dies muss ein Wert sein, der sonst nicht als Schlüssel vorkommen darf und sich auch von der „Frei“-Markierung unterscheiden muss.

Beim Einfügen von Elementen in die Tabelle können gelöschte Plätze wieder benutzt werden, das heißt gelöschte Plätze sind beim Einfügen wie leere Plätze zu behandeln. Beim Suchen müssen aber gelöschte Elemente aufgrund eventuell aufgetretener Kollisionsfälle fast wie normale Elemente behandelt werden, da ja bei noch nicht erfolgter Konfliktlösung „hinter“ ihnen noch Elemente stehen können, die zuvor mit den jetzt gelöschten Elementen kollidierten. Würde man einfach eine „Frei“-Markierung beim Löschen einsetzen, so würde die Suche an dieser „Frei“-Markierung stoppen, obwohl nach dem gelöschten Element noch weitere Kollisionsfälle liegen. Natürlich kommen die gelöschten Elemente selbst als Ergebnis des Suchlaufs nicht mehr in Betracht.

Falls die zu speichernden Elemente sehr umfangreich sind oder eine variable Länge haben, kann man in der Hashtabelle statt der Elemente auch nur Schlüssel und Pointer auf das jeweilige Element oder sogar nur Pointer auf Elemente speichern und die Elemente durch dynamische Speicherzuweisung (mit `malloc()`) erzeugen. Wenn nur Pointer verwendet werden, so könnte die „Frei“-Markierung beispielsweise der NULL-Pointer sein, während die Löschmarkierung ein Pointer auf ein einziges und eindeutiges „Löschelement“ sein könnte.

Es gibt auch andere Kollisionsbehandlungen als die lineare Sondierung. Beispielsweise muss das sondierte Element nicht zwingendermaßen das nächste im Array sein, sondern der Abstand zwischen den sondierten Elementen kann genauso wie der Schlüssel selbst abhängig vom tatsächlich zu speichernden Wert sein. Je nach Typ des Schlüssels und der erwarteten Verteilung können verschiedene Verfahren zu einer guten Verteilung innerhalb des Arrays führen.

Die Vielzahl der Möglichkeiten, Kollisionen innerhalb des Arrays aufzulösen, sprengt den Rahmen dieses Buches bei weitem, weswegen nicht weiter darauf eingegangen wird. Im Kapitel [→ 20.3.6](#) wird jedoch noch auf die Möglichkeit eingegangen, Konflikte mittels Elemente zu lösen, welche sich nicht mehr innerhalb des Arrays befinden.

### 20.3.4 Programmbeispiel für die Programmierung einer Hashtabelle mit linearer Sondierung

Das folgende Beispiel errechnet den Hash-Schlüssel eines Strings gemäß der folgenden Formel:

$$\left( \sum_{i=0}^{i < k} \text{asciiCode}(c_i) \right) \bmod M$$

Hierbei werden die einzelnen ASCII-Codes der Zeichen aufaddiert und dann diese Summe modulo der Tabellengröße geteilt. Jetzt das Programm:

hashtab.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define FREI      "0"      // Festlegung eines FREI-Zeichens
#define GELOESCHT "1"      // Festlegung eines GELOESCHT-Zeichens

typedef struct Messwert {
    char name[40];
    double wert;
} Messwert;

#define ANZAHL 8
Messwert tabelle[ANZAHL];
```

```
int hash(const char* name) {
    int summeASCII = 0;
    const char* ptrName = name;
    while (*ptrName != '\0') {
        summeASCII = (int)*ptrName + summeASCII;
        ++ptrName;
    }
    return (summeASCII % ANZAHL);
}

int rehash(int hash) {
    return (hash + 1) % ANZAHL;
}

void einfuegen(const char* name, double wert) {
    int z = 0;
    int index = hash(name);
    while ((strcmp(tabelle[index].name, FREI)) &&
           (strcmp(tabelle[index].name, GELOESCHT)))
    {
        index = rehash(index);
        ++z;
        if (z == ANZAHL) {
            printf("Tabelle voll\n");
            exit(1);
        }
    }

    // Freier Eintrag gefunden
    strcpy(tabelle[index].name, name);
    tabelle[index].wert = wert;
}

int sucheSchluessel(const char* name) {
    int z = 0;
    int index = hash(name);
    while (strcmp(tabelle[index].name, name)) {
        index = rehash(index);
        ++z;
        if (!strcmp(tabelle[index].name, FREI) || z == ANZAHL) {
            return -1;
        }
    }
    return index;
}
```

```
void loescheSchluessel(const char* name) {
    int index = sucheSchluessel(name);
    if (index != -1) {
        strcpy(tabelle[index].name, GELOESCHT);
        tabelle[index].wert = 0.;
    }else{
        printf("Schluessel %s nicht in Tabelle\n", name);
    }
}

static void initTabelle() {
    for (int i = 0; i < ANZAHL; ++i) {
        strcpy(tabelle[i].name, FREI);
        tabelle[i].wert = 0.;
    }
}

void ausgebenTabelle(void) {
    printf("\n");
    for (int i = 0; i < ANZAHL; ++i) {
        if (!strcmp(tabelle[i].name, FREI)) {
            printf("Index %d: FREI\n", i);
        } else if (!strcmp(tabelle[i].name, GELOESCHT)) {
            printf("Index %d: GELOESCHT\n", i);
        } else {
            printf("Index %d: %s (%d) = %1.02f\n",
                i,
                tabelle[i].name,
                hash(tabelle[i].name),
                tabelle[i].wert);
        }
    }
    printf("\n");
}

int main(void) {
    initTabelle();

    einfuegen("Temperatur", 23.);
    einfuegen("Luftfeuchtigkeit", 64.);
    einfuegen("Luftdruck", 1022.);
    einfuegen("Gewicht", 483.);
    einfuegen("Pegelstand", 644.);

    ausgebenTabelle();
}
```

```
int gewichtIndex = sucheSchluessel("Gewicht");
printf("Gewicht gefunden an Index %d\n", gewichtIndex);
int feuchtIndex = sucheSchluessel("Luftfeuchtigkeit");
printf("Luftfeuchtigkeit gefunden an Index %d\n", feuchtIndex);

loescheSchluessel("Luftfeuchtigkeit");
printf("Luftfeuchtigkeit geloescht.\n");

feuchtIndex = sucheSchluessel("Luftfeuchtigkeit");
printf("Luftfeuchtigkeit gefunden an Index %d\n", feuchtIndex);

ausgebenTabelle();

return 0;
}
```

Folgendes ist der Programmablauf:

```
Index 0: Pegelstand (7) = 644.00
Index 1: Temperatur (1) = 23.00
Index 2: FREI
Index 3: Gewicht (3) = 483.00
Index 4: Luftdruck (4) = 1022.00
Index 5: FREI
Index 6: FREI
Index 7: Luftfeuchtigkeit (7) = 64.00

Gewicht gefunden an Index 3
Luftfeuchtigkeit gefunden an Index 7
Luftfeuchtigkeit geloescht.
Luftfeuchtigkeit gefunden an Index -1

Index 0: Pegelstand (7) = 644.00
Index 1: Temperatur (1) = 23.00
Index 2: FREI
Index 3: Gewicht (3) = 483.00
Index 4: Luftdruck (4) = 1022.00
Index 5: FREI
Index 6: FREI
Index 7: GELOESCHT
```

### 20.3.5 Weitere Schlüsseltransformationen

Geeignete Schlüsseltransformationen zur Berechnung der ersten Position innerhalb der Tabelle mit Hilfe einer Hash-Funktion hängen stark von der Beschaffenheit des Schlüssels ab.



Es gibt viele Verfahren von Schlüsseltransformationen. Das in diesem Kapitel dargestellte einfache Verfahren (Index =  $x$  modulo  $M$ ) arbeitet – bei einigermaßen gleichförmig verteilten Schlüsseln und wenn  $M$  eine Primzahl ist – recht gut. Man erreicht dadurch meist eine gute Verteilung über die ganze Tabelle.  $M$  muss aber nicht eine Primzahl sein [Wet80].

Andere Verfahren selektieren einige Bits aus dem Schlüssel zur Berechnung des Hash-Index. Häufig bestehen die Schlüssel aus Strings, beispielsweise bei Symboltabellen in Compilern oder bei Namenslisten. So kann für Namen die erste Tabellenposition beispielsweise folgendermaßen berechnet werden:

Der Namensstring bestehe der Einfachheit halber nur aus Großbuchstaben und aus maximal  $n$  Zeichen.

Name =  $z_0 z_1 z_2 z_3 \dots z_{n-1}$

Den Namen kann man sich als variabel lange  $n$ -stellige Zahl zur Basis 27 (26 Buchstaben des Alphabets, 0 für Leerzeichen) vorstellen und daraus je unterschiedlichem Namen eine eindeutige Zahl erzeugen. Wenn man dem Buchstaben „A“ den Wert 1, dem Buchstaben „B“ den Wert 2, ... und „Z“ den Wert 26 zuordnet, so kann man daraus eine eindeutige Schlüsselzahl berechnen:

$$x = z_0 \cdot 27^{n-1} + z_1 \cdot 27^{n-2} + z_2 \cdot 27^{n-3} + z_3 \cdot 27^{n-4} + \dots + z_{n-1} \cdot 27^0$$

So liefert beispielsweise der Name „ABC“ die Zahl  $x = 1 \cdot 27^2 + 2 \cdot 27^1 + 3 \cdot 27^0 = 786$

Bei beliebig großen Namenslängen könnten sehr große Zahlen entstehen. Um die zugeordneten Schlüsselzahlen dann auf einen bestimmten Wertebereich (beispielsweise 32-Bit Zahlen) einzuschränken, kann man die Umrechnung auf die ersten 6 Buchstaben des Namens einschränken und zusätzlich die Namenslänge  $n$  mit in die Umrechnung einbeziehen. Die Zuordnung ist dann nicht mehr eindeutig, das heißt ein Kollisionsverfahren ist erforderlich.

Damit können den Namen beispielsweise folgende Schlüsselzahlen zugeordnet werden:

$$\text{Schlüsselzahl } x = z_0 \cdot 27^5 + z_1 \cdot 27^4 + z_2 \cdot 27^3 + z_3 \cdot 27^2 + z_4 \cdot 27^1 + z_5 \cdot 27^0 + n;$$

Mit dieser Berechnung ergeben sich Schlüsselzahlen, die kleiner sind als  $27^6 - 1 + n$ . Diese Werte sind selbst bei großen Namenslängen  $n$  deutlich kleiner als  $2^{32}$ .

Folgendes ist ein Beispiel, wie so eine Hash-Funktion ausprogrammiert sein könnte:

hashName.c

```
#include <string.h>
#include <stdio.h>
#include <ctype.h>

int hashName(const char* name) {
    int zahl = toupper(name[0]) - 'A' + 1;
    int len = (int)strlen(name);
    if (len > 6) len = 6;
    for (int i = 1; i < len; ++i) {
        if (isalpha(name[i]))
            zahl = zahl * 27 + toupper(name[i]) - 'A' + 1;
    }

    return zahl + (int)strlen(name);
}

int main(void) {
    printf("Hash: %d\n", hashName("C Programmieren"));
    printf("Hash: %d\n", hashName("D Programmieren"));
    return 0;
}
```

Das Ergebnis dieses Programmes lautet:

```
Hash: 1922800
Hash: 2454241
```

Es gibt noch viel mehr Möglichkeiten, Hash-Schlüssel zu definieren. Hier wird nicht weiter darauf eingegangen.

### 20.3.6 Direkte Verkettung bei Kollisionen

Bei den bisher dargestellten Verfahren der Konfliktlösung mussten die Tabellen ausreichend groß sein, um Platz für alle Elemente zu haben. Die Konflikte wurden innerhalb der Tabelle gelöst.

Hashverfahren, bei denen die Konflikte innerhalb der zur Verfügung gestellten Tabelle aufgelöst werden, werden auch als geschlossene Hashverfahren bezeichnet. Hashverfahren, bei denen im Kollisionsfall neue Elemente in einem zur Hashtabelle externen Bereich untergebracht werden, heißen offene Hashverfahren.



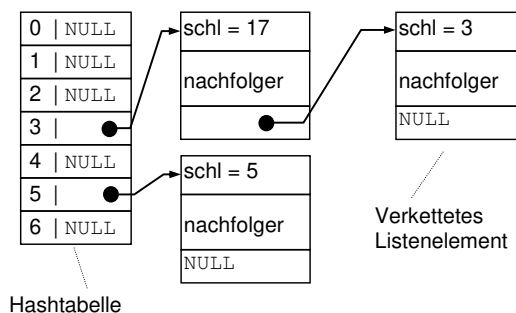
Ein Beispiel eines offenen Verfahrens ist die Verwendung einer einfach verketteten Liste, genannt „Direkte Verkettung“.

Bei der direkten Verkettung wird bei Kollisionen für die Elemente mit gleichem Tabellenindex eine verkettete Liste außerhalb der Tabelle angelegt, die dann linear verlängert und beim Suchen linear durchsucht wird.



Bei dieser Organisation der Hashtabelle reicht es aus, dass die Hashtabelle nur Pointer auf die Elemente enthält. Die Elemente selbst enthalten den Schlüssel, die restlichen Daten des Elementes sowie einen Verkettungs-Pointer zum Aufbau einer verketteten Liste im Kollisionsfall.

Das folgende Beispiel zeigt eine Hashtabelle mit verketteter Liste als Konfliktlösungsstrategie:





Die Position ergebe sich zu  $\text{Position} = x \text{ modulo } M$ , die Tabellenlänge sei  $M = 7$ . Es sollen drei Elemente mit den Schlüsseln 17, 3 und 5 eingetragen werden. Der Schlüssel 17 ergibt eine Position von 3, der nachträglich einzutragende Schlüssel 3 ergibt ebenfalls eine Position von 3. Dieser Konflikt wird durch Aufbau einer verketteten Liste für alle Elemente mit der gleichen Position (hier 3) gelöst. Bei diesem Verfahren müssen alle Array-Elemente der Hashtabelle zu Anfang mit dem NULL-Pointer belegt werden. Wird für eine Position ein Element eingetragen, so muss der Pointer an der Position der Hashtabelle auf dieses Element zeigen. Das Ende der verketteten Liste für Elemente mit gleichem Schlüssel wird durch den NULL-Pointer in diesem Element angezeigt. Wird für diese Position der Hashtabelle ein weiteres Element eingetragen, wird es an diese Liste angehängt.

Das Suchen geschieht in der Weise, dass zunächst aus dem Suchbegriff die Tabellenposition berechnet wird. Anschließend müssen im ungünstigsten Fall alle an der Position vorhandenen Listenelemente untersucht werden, bis das Element durch einen direkten Schlüsselvergleich gefunden wird. Beim Löschen muss man zunächst das Element suchen und anschließend aus der verketteten Liste entfernen.

Das folgende Programm zeigt, wie eine solche Hashtabelle erstellt werden kann. Die Löschoption wird jedoch ausgelassen.

hash.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct Element {
    int schluessel;
    struct Element* nachfolger;
} Element;

#define ANZAHL 11
Element* tabelle[ANZAHL];

void initTabelle(void) {
    for (int i = 0; i < ANZAHL; ++i)
        tabelle[i] = NULL;
}

int hash(int schluessel) {
    return (schluessel % ANZAHL);
}
```

```
void einfuegen(int schluessel) {
    int index = hash(schluessel);

    Element* neuesElement = malloc(sizeof(Element));
    neuesElement->schluessel = schluessel;
    neuesElement->nachfolger = NULL;

    if (tabelle[index] == NULL) {
        // Neues Element in die Tabelle einfuegen.
        tabelle[index] = neuesElement;
    } else {
        Element* element = tabelle[index];

        // Suche nach dem letzten Element
        while (element->nachfolger)
            element = element->nachfolger;

        // Neues Element in die Liste einfuegen.
        element->nachfolger = neuesElement;
    }
}

Element* sucheElement(int schluessel) {
    int index = hash(schluessel);
    Element* element = tabelle[index];

    while (element) {
        if (element->schluessel == schluessel)
            return element;
        element = element->nachfolger;
    }
    return element;
}

void ausgebenTabelle(void) {
    printf("Index | Schluessel\n");
    for (int i = 0; i < ANZAHL; ++i) {
        const Element* ptr = tabelle[i];
        while (ptr != NULL) {
            printf(" %2d | %5d\n", i, ptr->schluessel);
            ptr = ptr->nachfolger;
        }
    }
}
```

```
int main(void) {
    initTabelle();
    einfuegen(25);
    einfuegen(16);
    einfuegen(85);
    einfuegen(3);
    ausgebenTabelle();

    Element* element = sucheElement(3);
    if (element) {
        printf("Element gefunden: %d\n", element->schluessel);
    } else {
        printf("Element nicht gefunden.\n");
    }

    return 0;
}
```

Die Ausgabe des Programmes lautet:

| Index |  | Schluessel |
|-------|--|------------|
| 3     |  | 25         |
| 3     |  | 3          |
| 5     |  | 16         |
| 8     |  | 85         |

Element gefunden: 3

### 20.3.7 Vor- und Nachteile der Hashverfahren

Je höher der Füllgrad ( $n/M$ ) einer Hashtabelle ist, umso höher ist die Wahrscheinlichkeit einer Kollision.

Grundsätzlich ist zu vermeiden, dass die Hashtabellen zu voll werden, da dann bei allen Kollisionsbehandlungen der Tabellenzugriff sehr ineffizient wird.



Bei guten Verfahren – also guter Streuung der ersten berechneten Position und verschiedenen Indexfolgen bei Kollisionen – können Hashtabellen bis zu einer Auslastung von 80% bis 90% noch effizient arbeiten.

Wichtig bei allen Hashverfahren ist eine gute erste Schlüsseltransformation, welche die verschiedenen vorkommenden Schlüssel möglichst gleichförmig über die Hashtabelle verteilt.



Bei extrem schlechter Schlüsseltransformation – wenn beispielsweise alle Transformationen zur gleichen Position führen – nähert sich das Hashverfahren im Aufwand dem ineffizienten sequenziellen Suchen.



Löschoperationen können dazu führen, dass die Speicherstruktur nicht mehr ganz so effizient arbeitet. Einem zu löschenden Element ist grundsätzlich nicht anzumerken, ob andere Elemente mit diesem Element in Konflikt stehen und welche somit mittels Sondierung „nach“ dem zu löschenden Element eingefügt wurden. Aus diesem Grund kann bei einer Löschoperation das zu löschende Element nicht einfach entfernt werden, sondern muss durch eine Löschmarke ersetzt werden. Nur so kann gewährleistet werden, dass nachfolgende Suchoperationen andere Elemente, welche mit dem gelöschten Element in Konflikt standen, noch auffinden können. Bei Kollisionsauflösung mit direkter Verkettung ist dies kein Problem. Hier können gelöschte Elemente durch korrektes Setzen der Pointer einfach entfernt werden, ohne die Effizienz beim Suchen zu beeinträchtigen.

Ein weiterer Nachteil der Hashverfahren gegenüber anderen Suchmethoden ist der höhere Speicherbedarf, da Hashverfahren nur bei nicht voll besetzten Tabellen noch effizient arbeiten.

Man sieht an dieser Stelle wieder den sogenannten Time-Space-Tradeoff, einen der häufigsten Zielkonflikte beim Programmieren. Der Konflikt besteht darin, dass man eine Berechnung meist nur auf Kosten des Speicherverbrauchs beschleunigen kann, beziehungsweise dass man eingesparten Speicherplatz mit einer erhöhten Rechenzeit erkauft.

Bei den geschlossenen Verfahren muss die Größe der Tabelle vorher bekannt sein. Dies ist der Hauptnachteil dieser Verfahren. Wenn hingegen ein offenes Hashverfahren – beispielsweise mit direkter Verkettung – verwendet wird, ist eine variable Anzahl an Elementen möglich. Der Nachteil dabei ist aber, dass bei Häufungen der primären Positionen (verschiedene Schlüssel führen zu gleicher Position) die Effizienz sehr stark zurückgeht.

## 20.4 Suchen mit Hilfe von Backtracking

Nachdem am Beispiel des Sortierens zu sehen war, dass sich die Rekursion gut zum Erstellen von Algorithmen nach dem Prinzip „**teile und herrsche**“ eignet und auf diese Art leistungsfähige Algorithmen gebaut werden können, soll hier ein weiteres Problemlösungsprinzip beschrieben werden, das auf Rekursion beruht: das sogenannte „Backtracking“.

**Backtracking** beruht auf dem Prinzip „Versuch und Irrtum“: Es wird eine Möglichkeit ausgewählt und geprüft, ob der durch diese Auswahl eingeschlagene Weg zur Lösung führt. Wenn nicht, wenn also der Weg in einer Sackgasse endet, muss bis zu einer Stelle zurückgekehrt werden, bei der eine andere Möglichkeit ausgewählt und geprüft werden kann. Das englische Wort „to backtrack“ bedeutet auf Deutsch „zurückverfolgen“.



Viele wirtschaftliche und wissenschaftliche Probleme werden durch Backtracking-Algorithmen bearbeitet. Das Suchen optimaler Leiterplatten-Layouts und die optimale Routen- und Tourenplanung in einem Fuhrunternehmen seien hier als Beispiele genannt. Auch bei Strategiespielen wie beispielsweise Schach setzen Computer meist Backtracking-Algorithmen ein.



Wenn man bei der Suche an einem beliebigen Punkt steht, so hat man theoretisch 4 Möglichkeiten weiterzulaufen: nach Oben, Unten, Rechts oder Links. Entscheidet man sich für eine dieser 4 Möglichkeiten, so markiert man die derzeitige Zelle, die man soeben besucht hat, mit einem Zeichen, beispielsweise dem '0', um zu vermeiden, dass man später im Labyrinth wieder darüber stolpert und somit nicht im Kreis sucht.

Hat man den gewählten Schritt dann getätigt, nimmt man die damit erreichte Position als neue Startposition und setzt von da aus die Suche rekursiv fort. Man entscheidet sich also von dort aus wieder für eine der 4 Richtungen und macht den nächsten Schritt. Dies kann mittels eines rekursiven Aufrufs ein und derselben Funktion sehr einfach programmiert werden.

Nun kommt die zweite Grundidee des Backtrackings ins Spiel: Sobald ein Weg als nicht frei erkannt wird, wird die Rekursion gestoppt. Man muss wieder einen Schritt zurücktreten, sprich, an die aufrufende Funktion zurückkehren mit der Information, dass diese Richtung unbrauchbar war. Nachdem man den letzten Schritt rückgängig gemacht hat, also bei dem vorhergehenden Punkt steht, kann von dort aus eine der verbleibenden Möglichkeiten ausprobiert werden.

In dem hier vorgestellten Beispiel erfolgt das Zurückkehren zu einer vorherigen Situation – also das Backtracking – nicht explizit, sondern implizit beim Beenden eines rekursiven Aufrufs der Funktion.

Es gibt mehrere Gründe, warum eine Richtung unbrauchbar sein kann: Erstens kann in diese Richtung eine Wand (also ein #-Zeichen) sein. Zweitens kann man mit dem neuen Schritt eine Zelle besuchen, die man schon auf einem zuvor verfolgten Weg betreten hat, die also schon mit dem 0-Zeichen markiert wurde.

Die letzte Möglichkeit ist, dass sämtliche Richtungsmöglichkeiten (Oben, Unten, Rechts und Links) ausprobiert wurden und keine der Richtungen erfolgreich war. Auch dann wird die Rekursion gestoppt und an die aufrufende Funktion zurückgekehrt. Um den Fortschritt im folgenden Beispielprogramm sichtbar zu machen, wird die soeben besuchte Zelle jedoch noch mit 'X' markiert.

Die Suche hört erst auf, wenn man entweder einen Weg gefunden hat, der zum Ziel führt, oder aber keine der 4 Richtungen vom Startpunkt aus zum Ziel führen und somit der Algorithmus erfolglos zur `main()`-Funktion zurückkehrt.

Hier das Programmbeispiel:

laby.c

```
#include <stdio.h>

typedef enum Richtungen {
    RECHTS,
    OBEN,
    LINKS,
    UNTEN
} Richtung;

typedef enum Markierungen {
    WAND    = '#',
    FREI    = ' ',
    WEG     = 'O',
    IRRWEG  = 'X',
    ZIEL    = 'Z'
} Markierungen;

typedef enum SuchErgebnis {
    MISSERFOLG,
    ERFOLG
} SuchErgebnis;

#define ZEILEN 17
#define SPALTEN 25
char zellen[ZEILEN][SPALTEN] = {
    "#####",
    "# #      # #          #",
    "# # ### # # # # #####",
    "# # # # # # # # # #",
    "# # # # ### # ##### # # #",
    "# # # # # # # # # #",
    "# ##### ### ## #",
    "# # # # # # # # # #",
    "##### # ### # # ### # #",
    "# # # # # # # # # #",
    "# ### ##### # ##### ##",
    "# # # # # # # # # #",
    "##### ### # # # ### ## #",
    "# # # # # # # # # #",
    "# ##### # # # # # # #",
    "# # # # # # # #Z#",
    "#####",
};
```



```
void printLabyrinth(void) {
    for (int z = 0; z < ZEILEN; ++z) {
        for (int s = 0; s < SPALTEN; ++s) {
            printf("%c", zellen[z][s]);
        }
        printf("\n");
    }
    printf("\n");
}

SuchErgebnis suche(int z, int s) {
    if (zellen[z][s] == ZIEL) {
        return ERFOLG;
    }
    if (zellen[z][s] == FREI) {
        zellen[z][s] = WEG; // Weg markieren

        for (Richtung richtung = 0; richtung < 4; ++richtung) {
            SuchErgebnis ergebnis;
            switch (richtung) {
                case RECHTS: ergebnis = suche(z, s + 1); break;
                case OBEN:   ergebnis = suche(z - 1, s); break;
                case LINKS:  ergebnis = suche(z, s - 1); break;
                case UNTEN:  ergebnis = suche(z + 1, s); break;
            }
            if (ergebnis == ERFOLG)
                return ERFOLG;
        }

        zellen[z][s] = IRRWEG; // Misserfolg markieren
    }
    return MISSERFOLG;
}

int main(void) {
    SuchErgebnis ergebnis = suche(1, 1);
    printf(ergebnis == ERFOLG ? "Gefunden!\n" : "Meh.\n");
    printLabyrinth();
    return 0;
}
```

Hier die Ausgabe des Programms:

```
Gefunden!
#####
#0#00000#X#00000XXXXXXXXX#
#0#0###0#X#0# #0#####
#000# #0XX#0# #00000#XXX#
# # # #0###0# #####0#X#X#
# # #00000# #0XX#X#
# ##### #0###X#
# # # #000#X#X#
##### # ### # #0###X#X#
# # # # #00000#X#
# ### ##### # #####0###
# # # # #XXXXX#000#
##### ### # # #X###X###0#
# # # # #X#X#X#X#X#0#
# ##### # #X#X#X#X#0#
# # #XXX#XXX#Z#
#####
```

Von der Ausgangszelle `zellen[1][1]` ausgehend wird in diesem Beispiel in der `for`-Schleife zunächst in RECHTS-Richtung gesucht, also `suche(1,2)` aufgerufen. Da die Zelle `[1][2]` jedoch nicht frei, sondern mit dem Zeichen '#' belegt ist, wird sofort mit der Rückgabe `MISSERFOLG` zurückgewandert (Backtracking). Sodann wird in OBEN- und LINKS-Richtung gesucht, was ebenfalls in einer Wand endet.

Nach diesen drei Misserfolgen wird in die UNTEN-Richtung gewandert. Da die Zelle `[2][1]` frei ist, wird nun die Suche rekursiv fortgesetzt. Es wird erneut wieder in alle Richtungen gesucht, worauf man schlussendlich bei Zelle `[3][1]` landet. Dies wird solange fortgesetzt, bis das Ziel erreicht ist.

Etwas untypisch an diesem Beispiel für Backtracking ist dessen gutmütiges Laufzeitverhalten: Es ist proportional zur Anzahl der Zellen. Im Allgemeinen ist es jedoch so, dass Backtracking-Algorithmen exponentielles Laufzeitverhalten haben, da es nicht gelingt, hinreichend oft zurückzuwandern, also ganze Teile des zu durchsuchenden Raumes auszugrenzen. Ein Beispiel ist hier das Schachspiel, bei dem – wenn Rechner gegen Menschen antreten – enorme Rechnerleistungen eingesetzt werden, um dem exponentiellen Laufzeitverhalten Herr zu werden. Mehr zum Backtracking und dessen Analyse findet sich in [Sed92].

## 20.5 Übungsaufgaben

### Aufgabe 1: Bubblesort

Ein beliebtes (aber sehr ineffizientes) Suchverfahren ist das sogenannte „**Bubblesort**“. Bei diesem Verfahren steigen quasi die großen Werte ganz allmählich wie Luftblasen im Array nach oben, während die kleinen Werte langsam „auf den Grund sinken“. Das Verfahren führt folgende Schritte aus:

1. Starte beim Array-Index 0.
2. Vergleiche den Wert des aktuellen Index  $i$  mit dem Wert des Index  $i + 1$ .
3. Wenn der erste Wert größer ist als der zweite, vertausche die beiden Werte. Ansonsten muss nichts vertauscht werden.
4. Wiederhole Schritte 2 und 3 an der jeweils nächsten Position  $i + 1$ , solange, bis der Index  $n - 1$  erreicht wurde.
5. Starte daraufhin den gesamten Algorithmus erneut ab Schritt 1. Führe diese Wiederholung solange aus, bis in einem Durchlauf keine Vertauschungen mehr durchgeführt werden mussten.

Implementieren Sie diesen Algorithmus. Was sind die Vorteile und was sind die Nachteile gegenüber dem Sortieren mittels Selektion (siehe Kapitel [→ 20.1.1](#))?

### Aufgabe 2: Analyse der Vergleiche und Bewegungen

Betrachten Sie die Programme für das Sortieren mittels Selektion aus Kapitel [→ 20.1.1](#), das Programm für Quicksort aus Kapitel [→ 20.1.3](#) und das Bubblesort-Programm aus der Übungsaufgabe 20.1.

- Erweitern Sie die Programme so, dass zu Beginn des Programmes das Array mit einer beliebig großen Anzahl  $n$  Elementen angelegt und automatisch mit zufälligen Werten gefüllt wird. Nutzen Sie hierfür die `malloc()`- und `rand()`-Funktionen der `<stdlib.h>`-Bibliothek.
- Erweitern Sie die Programme, sodass die Anzahl Vergleiche und Bewegungen in einer globalen Variable gezählt, und am Ende des Algorithmus ausgegeben werden.
- Testen Sie die Algorithmen mit verschiedenen Array-Größen und machen Sie sich einen Überblick, wie sich die verschiedenen Algorithmen verhalten.

### Aufgabe 3: Löschen bei Hashtabellen mit Liste

Im Kapitel [→ 20.3.6](#) wurde das Hashverfahren mithilfe einer linearen Liste vorgestellt. Erweitern Sie die Programmierung, sodass Elemente auch wieder gelöscht werden können.

### Aufgabe 4: Lösen von Sudokus mit Backtracking

Wie zum Finden von Wegen im Labyrinth eignet sich Backtracking auch sehr gut, um Sudoku-Rätsel zu lösen. Ein Sudoku sieht zum Beispiel so aus:

|   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|
|   |   |   | 5 | 4 | 2 |   |   |   |
| 9 | 6 | 7 |   |   |   |   |   |   |
|   |   |   |   |   |   | 3 | 1 | 8 |
|   |   |   |   | 7 |   | 8 | 6 | 4 |
|   | 2 |   | 6 |   | 4 |   | 9 |   |
| 6 | 4 | 5 |   | 1 |   |   |   |   |
| 8 | 9 | 1 |   |   |   |   |   |   |
|   |   |   |   |   |   | 5 | 4 | 7 |
|   |   |   | 3 | 2 | 6 |   |   |   |

Die Regeln für die korrekte Lösung sind einfach: Die leeren Felder in der 9×9-Matrix sollen so mit den Ziffern 1...9 gefüllt werden, dass ...

- ... in keiner Zeile eine Ziffer doppelt vorkommt.
- ... in keiner Spalte eine Ziffer doppelt vorkommt.
- ... in keinem der fett gedruckten 3×3 Blöcke eine Ziffer doppelt vorkommt.

Ziel der Aufgabe ist es, mittels eines Backtracking-Algorithmus die Lösung für ein 9×9-Sudoku zu berechnen.

Der Backtracking-Algorithmus soll versuchen, in jeder noch nicht gefüllten Zelle die Werte 1...9 durchzuprobieren. Die Prüfung, ob es sich dabei um einen gültigen Wert handelt, ist etwas komplizierter als beim Labyrinth, bei dem nur geprüft werden musste, ob die Zelle frei ist. Hier bietet es sich an, zunächst Funktionen auszuprogrammieren, welche prüfen, ob der einzutragende Wert in einer Zeile, einer Spalte und in einem Block überhaupt erlaubt ist:

```
int istZeileOK (int zeile, char wert);
int istSpalteOK(int spalte, char wert);
int istBlockOK (int zeile, int spalte, char wert);
```

Diese drei Funktionen müssen alle ihr OK geben, bevor der Algorithmus rekursiv versucht, einen Wert 1 . . . 9 einzutragen. Wenn alle Ziffern 1 . . . 9 erfolglos ausprobiert wurden wird die Suche abgebrochen und in der aufrufenden Suche die nächste mögliche Zahl ausprobiert.

Das Sudoku und auch dessen Lösung sollen dabei wie das Labyrinth in einem zweidimensionalen char-Array abgelegt werden:

```
char zellen[9][9] = {
    "  542  ",
    "967    ",
    "      318",
    "    7 864",
    " 2 6 4 9 ",
    "645 1    ",
    "891     ",
    "      547",
    "   326   ",
};
```

Die Lösung sollte folgendermaßen aussehen:

```
+ ----- + ----- + ----- +
| 1 8 3 | 5 4 2 | 6 7 9 |
| 9 6 7 | 1 8 3 | 4 5 2 |
| 4 5 2 | 7 6 9 | 3 1 8 |
+ ----- + ----- + ----- +
| 3 1 9 | 2 7 5 | 8 6 4 |
| 7 2 8 | 6 3 4 | 1 9 5 |
| 6 4 5 | 9 1 8 | 7 2 3 |
+ ----- + ----- + ----- +
| 8 9 1 | 4 5 7 | 2 3 6 |
| 2 3 6 | 8 9 1 | 5 4 7 |
| 5 7 4 | 3 2 6 | 9 8 1 |
+ ----- + ----- + ----- +
```

**Zusatzaufgabe:** Wenn Sie die drei Ziffern in der ersten Zeile weglassen, gibt es insgesamt 3 mögliche Lösungen. Bringen Sie ihr Programm dazu, alle drei Lösungen auszugeben.